5.1 Automatyzacja Pipeline'ów: Wykorzystanie LLM do generowania i naprawy plików konfiguracyjnych YAML (GitHub Actions)

Moduł 5: Agenty AI w procesach CI/CD i DevOps

Krzysztof Rytlewski · 2026

CI/CD i GitHub Actions — pojęcia podstawowe

CI/CD (Continuous Integration / Continuous Deployment) to praktyka automatycznego budowania, testowania i wdrażania kodu przy każdej zmianie w repozytorium — eliminuje ręczne kroki i minimalizuje ryzyko błędów ludzkich.

Pojęcie	Opis
Workflow	Cały plik YAML definiujący pipeline; wyzwalany przez zdarzenie
Event	`push`, `pull_request`, `schedule`, `workflow_dispatch`
Job	Niezależna jednostka pracy uruchamiana na osobnym runnerze
Step	Pojedynczy krok w jobie — komenda shell lub gotowa akcja
Action	Reużywalny krok z GitHub Marketplace, np. `actions/checkout@v4`
Secret	Zmienna środowiskowa zaszyfrowana w ustawieniach repozytorium

Pliki YAML mają ścisłą hierarchię: `on` → `jobs` → `steps`. Błąd wcięcia lub literówka psuje cały pipeline.

Dlaczego YAML pipeline'ów jest trudny?

Pisanie plików GitHub Actions ręcznie ma kilka pułapek:

- **Składnia YAML** jest wrażliwa na wcięcia — spacje vs tabulatory powodują niejasne błędy.
- **Nazwy akcji i wersje** (`actions/checkout@v4`, `actions/setup-node@v4`) zmieniają się z czasem.
- **Wyrażenia kontekstowe** ($secrets.API_KEY$‘,‘`{{ github.sha }}`) mają nieintuicyjną składnię.
- **Macierze testów** (`strategy.matrix`) wymagają precyzyjnej konfiguracji.
- Błędy są widoczne dopiero po uruchomieniu pipeline'u — pętla feedbacku jest wolna.

To właśnie miejsce, gdzie LLM są wyjątkowo pomocne — generują, naprawiają i optymalizują YAML szybciej i dokładniej niż ręczna edycja.

Co może LLM w kontekście GitHub Actions?

LLM (np. przez GitHub Copilot lub OpenAI API) mogą w kontekście CI/CD:

1. **Generować** pełny plik workflow z opisu w języku naturalnym — oszczędność godzin konfiguracji.
2. **Naprawiać** błędne pliki na podstawie komunikatu o błędzie z logu Actions.
3. **Optymalizować** istniejące pipeline'y — dodawać cache, równoległość między jobami, warunkowe uruchamianie.
4. **Tłumaczyć** skomplikowane wyrażenia na prostszy kod lub objaśniać istniejącą konfigurację.

Kluczową umiejętnością jest konstruowanie efektywnych promptów zawierających kontekst: stack technologiczny, framework, oczekiwane kroki pipeline'u i wersje narzędzi.

Generowanie pipeline'ów przez LLM — jak tworzyć skuteczne prompty

Dobry prompt do generowania YAML GitHub Actions zawiera:

- cel pipeline'u (test, build, deploy),
- stack technologiczny (Node.js, Angular, AWS Lambda) z wersjami,
- oczekiwane kroki w odpowiedniej kolejności,
- warunki wyzwolenia (`push` na `main`, PR do `main`).

Przykładowy prompt:

> *„Wygeneruj workflow GitHub Actions dla backendu Node.js 22 i frontendu Angular 19. Pipeline powinien: uruchamiać testy backendu przy każdym PR do `main`; budować Angular produkcyjnie i wgrywać do S3 przy `push` na `main`; aktualizować kod AWS Lambda przy `push` na `main`. Używaj `actions/cache` dla `node_modules`. Klucze AWS jako GitHub Secrets."*

Z takiego promptu LLM generuje gotowy workflow — twoja rola to weryfikacja i dostosowanie nazw zasobów.

Przykład wygenerowanego fragmentu z cache'm

```yaml
jobs:
  test-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '22'
          cache: 'npm'
          cache-dependency-path: backend/package-lock.json

      - name: Install & test
        working-directory: backend
        run: |
          npm ci
```

Parametr `cache: 'npm'` w `actions/setup-node` automatycznie cachuje `node_modules` między uruchomieniami — oszczędność ~60 sekund per run.

Automatyczna naprawa błędów pipeline'u przez LLM

Gdy pipeline kończy się błędem, GitHub Actions dostarcza szczegółowy log. Przekazanie tego logu do LLM wraz z plikiem YAML pozwala na automatyczną diagnozę i naprawę.

Wzorzec „diagnoza przez LLM":

1. Pobierz log błędu z GitHub API (przez Octokit lub ręcznie z zakładki Actions).
2. Wyślij log + zawartość pliku YAML do OpenAI API z prośbą o naprawę.
3. Otrzymaj poprawiony YAML + wyjaśnienie zmian.
4. Utwórz commit lub PR z poprawką.

Skrypt Node.js (`fix-pipeline.js`) automatyzuje kroki 1–3: pobiera log przez `octokit.actions.downloadWorkflowRunLogs()`, wysyła go do `gpt-4o` i zapisuje poprawiony plik. Całość może działać jako krok w GitHub Actions wyzwalanym zdarzeniem `workflow_run` na nieudanym runie.

Strategie macierzowe — równoległe testowanie wielu konfiguracji

Macierze testów (`strategy.matrix`) pozwalają uruchamiać ten sam job z różnymi konfiguracjami jednocześnie — np. Node.js 20 i 22 z MongoDB 6.0 i 7.0 (4 kombinacje równolegle).

```
strategy:
  matrix:
    node-version: [20, 22]
    mongodb-version: ['6.0', '7.0']
  fail-fast: false   # nie przerywaj przy pierwszym błędzie
services:
  mongodb:
    image: mongo:${{ matrix.mongodb-version }}
    ports:
      - 27017:27017
```

`fail-fast: false` pozwala zebrać wyniki wszystkich kombinacji — bez tego jedno niepowodzenie zatrzymałoby pozostałe, ukrywając inne potencjalne błędy. LLM może wygenerować pełną konfigurację macierzy z opisem wymagań.

Typowe optymalizacje pipeline'ów sugerowane przez LLM

- **Cache `node_modules`** przez `actions/cache` lub wbudowany cache w `setup-node` — oszczędność ~60 sek. per run.
- **Podział na równoległe joby** — backend i frontend budowane jednocześnie zamiast sekwencyjnie, skrócenie całkowitego czasu pipeline'u.
- **Warunkowe wdrożenie** — `if: github.ref == 'refs/heads/main'` uruchamia deploy tylko dla gałęzi `main`, nie dla PR.
- **Wyzwalanie po ścieżkach** — `paths: ['frontend/**']` uruchamia job frontendu tylko gdy zmieniły się pliki w `frontend/`, pomija go przy zmianach tylko w backendzie.
- **Przypinanie wersji akcji** do konkretnych SHA zamiast tagów — ochrona przed przestawieniem tagu przez zewnętrznego dostawcę.

Bezpieczeństwo pipeline'ów CI/CD

Pipeline CI/CD ma dostęp do sekretów produkcyjnych — błędna konfiguracja może prowadzić do ich wycieku.

Najważniejsze zasady:

- **Nigdy nie loguj sekretów** — `echo ${{ secrets.API_KEY }}` maskuje wartość jako `***` w logach, ale obejście przez podproces może ominąć maskowanie.
- **Ogranicz uprawnienia `GITHUB_TOKEN`** — deklaruj tylko `contents: read` i `deployments: write`, nie przyznawaj pełnych uprawnień.
- **Przypinaj akcje zewnętrzne do commitów** — tag może zostać przestawiony przez autora; SHA jest niezmienialny.
- **Ogranicz sekrety do jobów** — nie przekazuj wszystkich sekretów produkcyjnych do jobów testowych.
- **Rozróżniaj `pull_request` od `pull_request_target`** — PR z forków zewnętrznych mają ograniczony dostęp do sekretów.

LLM może przejrzeć istniejący plik YAML pod kątem tych zagrożeń i zasugerować poprawki.